

AI 使用教學 · 補充篇

Chat 與 Code

在結構設計裡，到底差在哪？

不談技術，只談「什麼時候用哪一個」

為什麼要分清楚這兩個？

用錯工具不會出錯，但會「事倍功半」——選對了，省下來的時間才看得到。

都是 Claude

兩個是同一個 AI 的不同入口，能力相通；差別在「你把它接到什麼東西上」。

新手先分這個

不必先學功能，先學「這件事該用哪一個」。判斷對了，上手就快。

今天的目標

看完這幾頁，拿到任何一件工作，你能在三秒內說出「這個用 Chat 還是 Code」。

先給結論

差別不在「誰比較聰明」，而在「對象是對話，還是你的檔案」。



Claude (Chat)

- 你在對話框裡「問跟答」
- 對象是一段文字、一個想法、你貼上來的內容
- 適合：想清楚、寫東西、整理判斷
- 一次處理一件事，看完即用



Claude Code (桌面版)

- 你把它指向「電腦裡的一堆檔案」
- 對象是整個資料夾、多份文件、重複流程
- 適合：批次處理、跨檔比對、產出檔案
- 幫你動手做，產生實際檔案

換個比喻就懂了

Claude (Chat)



像坐你旁邊的顧問

你問他、他答你。要靠你把資料說給他聽，他的回答你再自己拿去用。



一次一個主題

換個案子、換個檔案，就重開一段乾淨的對話。

Claude Code (桌面版)



像能進你檔案室的助理

你說「這個資料夾」，他自己打開、讀完、處理，再把成品放回去。



一次一整批

20 份廠商型錄、整季的會議紀錄，一次跑完。

關鍵差別：Chat 看「你貼給它的」；Code 看「你電腦裡的整批檔案」。

適合用 Chat 的場景

特徵：一次一件事、靠你提供內容、要的是「想法或一段文字」。



寫客戶往來信

貼上對方來信，請它草擬語氣得體、立場清楚的回覆，你再改。



整理會議紀錄

把零散筆記丟進去，請它整理成「決議 / 待辦 / 負責人」。



問規範觀念

「這種情況載重組合的邏輯是什麼」——當作懂結構的對話對象。



潤飾計算書敘述段

數字你自己跑；只請它把說明文字寫得更通順、更專業。



共同點：資料是「你給的一段」，產出是「你拿去用的一段」。它不會自己翻你的檔案，也不會幫你存檔。

Chat 實際長怎樣？一個對話示範

情境：把一段凌亂的會議筆記，請 Chat 整理成可用的紀錄。

你 ▶ 貼上筆記 + 一句指令

「幫我把下面這段會議筆記整理成：決議、待辦（含負責人）、待確認事項三塊。
6/2 結構會：地下室開挖深度待業主確認...老張負責回算...下週五前給圖...」

Claude ▶ 回你一段整理好的文字

【決議】開挖深度暫以 -8.5m 進行初步分析。

【待辦】老張：回算側向土壓力，6/6 前。 | 老周：更新地下室配筋圖。

【待確認】開挖深度需業主 6/4 前回覆，否則順延。

(你看過、改兩個字，就能貼進正式紀錄。)

這個示範的重點

- 資料是你貼進去的，不是它去翻檔案
- 它給你一段文字，存不存檔由你決定
- 指令講清楚「要哪三塊」，結果就好用
- 你只花時間「看與改」，不用從頭打字

適合用 Code 的場景

特徵：一整批檔案、重複性高、要的是「處理過的檔案成品」。



批次比對 20 份廠商型錄

指向放型錄的資料夾，請它逐份抽規格、整理成一張比較表。



整理整個案子的會議紀錄

把整季的紀錄檔丟給它，依日期彙整待辦與進度。



跨多份文件找一致性

「這幾十份檔案哪裡寫法不一致」——它逐檔讀完再回報。



重複套版的文件

同一份格式套到很多案子，請它依資料逐份產生檔案。



共同點：都是「同一動作做很多次」或「要翻很多檔案」。它會自己讀檔、處理、產出——也因此你更要檢查它做了什麼。

Code 實際長怎樣？一個任務示範

情境：一個資料夾裡有 20 份廠商型錄 PDF，要做一張規格比較表。

STEP 1

你給它資料夾

「讀這個資料夾裡所有型錄 PDF」——
你不用一份份打開。

STEP 2

它逐份讀完並抽資料

20 份一份份打開、找出載重、跨距、規格欄位。

STEP 3

它產出一個檔案

輸出一張整理好的 Excel 比較表，存回資料夾給你。

和 Chat 的差別在這裡

同樣這件事用 Chat：你得自己開 20 份、一份份貼進去、再自己彙整——做不完。
用 Code：你只下一句話，它自己讀完整批、產出檔案。重複越多次，省得越多。

拿到一件事，怎麼選？

三個問題，問完就知道用哪一個。

		Chat	Code
1	處理一件，還是一整批？	一件事、一段文字	一整批檔案
2	資料在對話裡，還是在檔案裡？	我直接貼給它	在電腦資料夾裡
3	我要想法，還是要檔案成品？	想清楚 / 一段文字	處理好的檔案

拿不定主意就先用 **Chat**。它門檻最低、最安全；等你發現「同一件事一直重複做」，再換 **Code**。

三個常見誤解

剛開始用最容易卡在這幾點，先講清楚。

常聽到的誤會

✗ Code 比較強，所以都用 Code 就好。

✗ 我不會寫程式，Code 跟我無關。

✗ AI 整理完就能直接用、不用看。

實際上是這樣

✓ 兩個能力相通。日常一件件事，Chat 反而更快、更省事

✓ Code 是用中文交代任務，不是你自己寫程式。它幫你動手

✓ 永遠要看。尤其非標準案例最容易出錯，數量級先檢查一遍。

不管用哪一個，這兩件事都一樣



不交給 AI 的事

- 結構計算的最終結論、載重判斷、安全認定
- 簽證與負責覆核——這是法律責任，AI 不能扛
- 把它當「初稿與助手」，不是「決策者」



一定要做的事

- 數量級檢查：結果差一個位數，一眼抓出來
- 可追溯：它的依據要查得到、對得上
- 非標準案例最容易出錯——這種更要自己盯

一句話記住

AI 負責「跑得快」，你負責「對得起簽名」。工具換了，責任沒換——這條線在結構這行不會變。

這週可以做的第一步

不用一次學會，挑一件小事開始就好。

1

挑一件「常做、出錯也沒風險」的事

例如整理會議紀錄、潤飾說明文字。不要拿關鍵計算當第一次練習。

2

先用 Chat 試三次

習慣「問 → 改 → 用」的節奏，感受它哪裡幫得上、哪裡要你把關。

3

發現一直重複、又得翻很多檔案，再認識 Code

到那時你自然會知道為什麼需要它——而不是現在硬學。

有問題隨時開一個對話問 Claude——它就是來陪你踩坑的。